

Система автоматизированного обогащения товарных данных для интернет-магазина

Фролов М. А.

Московский авиационный институт
Институт № 8, кафедра 806 «Вычислительная математика и программирование»
Научный руководитель: Судаков В. А., д. т. н., профессор кафедры 806

Москва, 2026

Две стороны проблемы обогащения

Человек не масштабируется

- ▶ Каталоги — десятки миллионов SKU
- ▶ Партнёры присылают только базовые поля
- ▶ Заполнение 50–150 атрибутов вручную = узкое место
- ▶ PIM хранит, но не извлекает

Машина-в-лоб слишком дорого

- ▶ Прямой вызов БЯМ на каждый SKU
- ▶ На 10^7 SKU — неприемлемая стоимость
- ▶ Точность vs цена — компромисс
- ▶ Нет контроля качества «на крае»

⇒ Нужно дешёвое решение, не теряющее качество — **поатрибутная маршрутизация** между слоями разной стоимости.

Аналоги: пробел в существующих подходах

Критерий	TXtract	MAVE	FrugalGPT	AutoMix	Наша система
Поатрибутная политика	✓	✓	—	—	✓
Калиброванный отказ от ответа	—	—	—	✓	✓
Маршрутизация по стоимости	—	—	✓	✓	✓
Контроль на непересекающихся товарах	—	—	—	—	✓
Гибридная архитектура слоёв	—	—	—	—	✓
Локальное развёртывание	частично	частично	—	—	✓

Ни одна из существующих систем не объединяет **поатрибутную политику**, **контроль на новых товарах** и **гибридную архитектуру** одновременно.

TXtract / MAVE (Amazon / Google): требуют GPU и трансформерной инфраструктуры.

Цель, задачи и рабочая гипотеза

Цель. Программное средство обогащения товарных данных на основе гибридной каскадной архитектуры.

Задачи

1. Архитектура гибридного каскадного конвейера (5 слоёв) с поатрибутной политикой.
2. Реализация каждого слоя как самостоятельного компонента с независимым обновлением.
3. Эталонная разметка и тестовая выборка без пересечения товарных кодов.
4. Экспериментальная оценка: точность, покрытие, стоимость; сравнение с прямой БЯМ.
5. Демонстрационный программный комплекс с REST API и веб-интерфейсом.

Гипотеза. Каскад `regex` $\rightarrow$ ML $\rightarrow$ Bayes $\rightarrow$ LLM с поатрибутной

Процесс обогащения карточки: BPMN

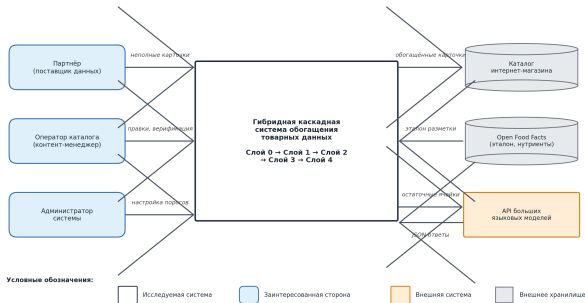
TODO: images/fig_bpmn_enrichment.png

BPMN 2.0: 4 дорожки (Партнёр / Сервис / Оператор / Каталог),
~15 узлов (события начала/конца, 5 задач, 2 шлюза XOR, потоки
управления и сообщений).

Создать в draw.io по §4.1 спеки.

BPMN 2.0 — диаграмма бизнес-процесса. Актёры (дорожки-swimlanes), задачи (прямоугольники со скруглениями), решения (ромбы-шлюзы XOR), события (окружности).
Схема алгоритма каскада с детализацией слоёв — слайд 7.

Архитектура системы (ГОСТ Р 57100-2016)



Контекстная диаграмма: заинтересованные стороны, исследуемая система, внешние хранилища

Потребности заинтересованных сторон

точность извлечения; стоимость единицы; время отклика; калибровка отказа; способность к развитию

Точки зрения

контекстная, функциональная, процессная, данных

Виды

контекст (слева), функция (B16),

Формальная задача и критерий качества

Задача:

$$f: X \rightarrow A \cup \{\perp\}, \quad \perp \text{ при } c < \tau_a.$$

X — пространство партнёрских карточек; A — атрибутная мульти-метка ($|A| = 21$); $\perp$ — отказ от ответа при низкой калиброванной уверенности.

Критерий:

$$Q = w_1 \text{Acc} + w_2 \left(1 - \frac{c_{\text{LLM}}}{c_{\text{max}}}\right) + w_3 \text{Cov} - w_4 \text{ECE}.$$

Бюджет $\beta = 10\%$, ECE-ограничение $\leq 0,05$.

Полная формализация и состав требований — §3.3 ВКР и backup-слайды B13 (требования), B0 (метрики).

Что делает каждый слой

21 классификатор по парам «категория-атрибут» (8/6/7); бюджет LLM $\beta = 10\%$, фактически **4,1 %**.

Слой	Что делает	Когда не справляется	Доля
L0 router	XGBoost на эмбединге → катего- рия + OOD-флаг	OOD ⇒ слой 4 «как есть»	—
L1 regex	Прецизионные поатрибутные шаблоны (precision $\approx 91\%$ на покрытых ячейках)	Нет матча ⇒ слой 2	21,9 %
L2 ML	XGBoost на 768-мерном эмбед- динге; Platt + isotonic	$p_{\max} < \tau_a \Rightarrow$ слой 3	73,2 %
L3 Bayes	Валидатор плотностей $P_B(\hat{y} \mid \epsilon)$	$P_B < \theta_a \Rightarrow$ слой 4	0,8 %
L4 LLM	gpt-oss-120b / Sonnet 4.5 / Gemini Flash + few-shot	— (последний слой)	4,1 %

⇒ 95 % ячеек закрываются дешёвыми слоями (L1+L2+L3); LLM — только на сложном остатке.

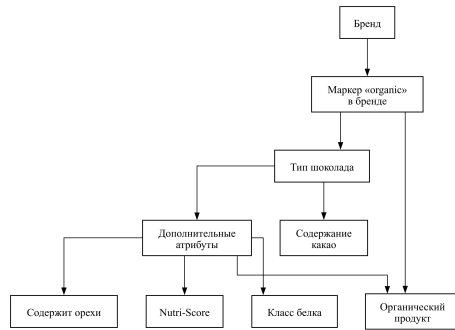
Слой 2 — эмбединги (82 275); XGBoost (мощь 51 0 840, ECF 0 026), MLP (0 857; 0 021)

Bayes — валидатор, не классификатор

Что построено. pgmpy 1.1.2: структура DAG
— Hill Climb + BIC; CPT —
BayesianEstimator с BDeu.

Почему не классификатор:

- ▶ самостоятельная точность $\approx 52\%$ (ниже порога тривиального большинства),
- ▶ но $P(\hat{y} | \epsilon)$ на нижнем 5-перцентиле даёт сигнал отбраковки,
- ▶ включён точно на **1 паре из 20** —
chocolate/contains_nuts: **+6,6 пп** точности
при **+5,3 пп** расходов на запасной слой.



DAG для категории chocolate

Данные и протокол оценки

Источник. Open Food Facts, 4,5 млн SKU (~40 % FR / 10 % EN / 7 % DE).

3 категории. Паста (8 атр.), шоколад (7), сыры (7) — **21 пара** «категория-атрибут».

Эталон. Два уровня: основной консенсус БЯМ ($n = 16\,360$) + проверочный ручной ($n = 566$, нижняя оценка).

Протокол. Разбивка без пересечения товарных кодов; предрегистрация `cd9ac7a` (2026-05-13). Порог уверенности τ_a — подобран на валидационной выборке, не на тестовой.

Языки. Без отдельных моделей на язык: данные ~40 % FR / 10 % EN / 7 % DE + IT, ES

Категория	Атрибутов
Паста	8
Шоколад	7
Сыры	7
<i>Холодный старт:</i>	
Электроника	8

Главный результат

Метрика	Значение	<i>n</i>
Точность каскада (микро, ДИ Вильсона)	95,5 % [95,2; 95,8]	16 360
Макро-F1 каскада (взвеш. по ячейкам)	0,890	16 360
Макро-precision / макро-recall	0,918 / 0,881	16 360
Точность маршрутизатора категории	97,2 %	570
Покрытие сквозной цепочки	97,3 %	16 360
Точность сквозной цепочки	93,0 % [92,6; 93,4]	16 360
Нижняя оценка на ручной разметке	86,7 % (каскад 91,3 %, F1 0,843)	566

Выборка без пересечения товарных кодов. **Доля языковой модели: 4,1 %** ячеек ⇒ архитектурное сокращение стоимости в **24×** раз (100 %/4,1 %; совокупно с заменой модели — в 580 раз).

Достоверность. ДИ Уилсона 95 %: **[95,2; 95,8] %** ($n = 16,360$, выборка без

Каскад vs прямая БЯМ

Что значит «прямая БЯМ»: одна и та же модель спрашивается на 100 % ячеек, без слоёв L1/L2/L3. Сравнение на одной выборке ($n = 16\,360$, покрытой каскадом).

Конфигурация	Точность	Стоимость	Дешевле в
Прямая Claude Sonnet 4.5 (опорная)	83,8 %	1,000	1×
Прямая gpt-oss-120b (открытая)	69,3 %	0,030	33×
Каскад + Gemini 2.5 Flash	93,0 %	0,0017	580×
Каскад + Claude Sonnet 4.5	92,9 %	0,041	24×
Каскад + gpt-oss-120b (локально)	90,6 %	0,0012	810×

Вклад слоёв: regex 21,9 % | ML 73,2 % | Bayes 0,8 % | LLM 4,1 % (~95 % ячеек закрываются дешёвыми слоями)

Демо: ~200 мс на запрос

Демо-комплекс.

- ▶ REST API (FastAPI) + веб-интерфейс
- ▶ Полный каскад L0–L4 за один HTTP-вызов
- ▶ Локальное развёртывание: `docker-compose up`
- ▶ Возврат: значение, слой-источник, уверенность

Латентность: ~200 мс
(cascade) / ≤ 5 с
(LLM-эскалация).

Обогащение товарных данных

Каскад: regex → ML → байес → LLM-fallback. Введите частично заполненный товар — система предскажет недостающие атрибуты.

Ввод товара

Режим определения категории

☒ Авто (классификатор) ☐ Вручную

Категория

Название товара *

Бренд / производитель

Состав

Количество

Примеры:

вайдиты:

с противоядами:

Результат

Категория:	Авто (router):	OOD:
chocolate	chocolate, p=0.97	нет
Закрито:	LLM-fallback:	Время ответа:
6 / 7 атрибутов	1 атрибут	184 мс

ПРЕДСКАЗАНИЯ ПО АТТРИБУТАМ

АТТРИБУТ	ПРЕДСКАЗАНИЕ	СЛОЙ	УВЕРЕННОСТЬ
cocoa_content_class	70-85 %	regex	1.00
chocolate_type	dark	regex	1.00
contains_nuts	false	ml	0.93
is_organic	false	ml	0.87
contains_milk	false	ml	0.81
flavor_modifier	vanilla	bayes	0.74
filling_type	— будет дозаполнено	llm	—

Слой каскада: regex ml bayes no-fallback

Рисунок 4.1 — Веб-интерфейс демонстрационной системы (схематично): форма ввода и блок результата с отметкой слоя-источника и оценкой уверенности

Веб-интерфейс демо-комплекса

Применение в интересах работодателя

**Ожидаемый эффект: снижение стоимости обогащения в
~24× (архитектурное) при точности ~95,5 % (95 % ДИ
Уилсона)**

Оператор каталога

Ручная проверка ~3–7 % ячеек
вместо 100 %. Дашборд качества.

Партнёр-поставщик

API-контракт без изменений.
Мгновенное обогащение карточки.

Разработана в интересах партнёрского интернет-магазина (название по запросу комиссии).

Администратор PIM

Контроль расходов на БЯМ.
`docker-compose up` — рядом с PIM.

Развитие системы

Новые домены через композицию схем.
Локальный резервный слой при
недоступности API.

Что нового в работе

Научная значимость.

- ▶ Методика построения гибридного каскада с **поатрибутной калиброванной уверенностью** и **предрегистрируемым** протоколом оценки.
- ▶ Предрегистрированный **отказ** от гипотезы об обучаемом маршрутизаторе в пользу статической политики (простота и предсказуемость).
- ▶ Воспроизводимость методики подтверждена на новой категории (электроника) в режиме холодного старта.

Практическая значимость.

- ▶ Работающее ПО в `docker-compose`, интегрируется рядом с PIM.
- ▶ Кратное сокращение стоимости и ручной нагрузки контент-менеджеров.
- ▶ Возможность локального развёртывания — независимость от внешних API.

Заключение

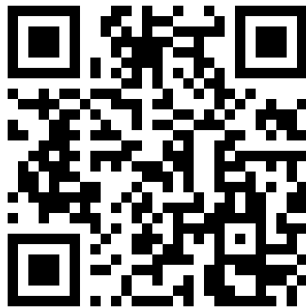
Задача	Результат
1. Архитектура каскада	5 слоёв с конъюнктивным AND-условием на пороги; явный отказ слоя; идентификация слоя-источника каждого предсказания.
2. Реализация слоёв	21 классификатор по 3 категориям (8/6/7); независимое обновление; <code>docker-compose</code> : Go-шлюз + Python/FastAPI ML-сервис + Web-UI.
3. Эталонная разметка	консенсус БЯМ $n=16\,360$ + ручной $n=566$; разбивка без пересечения кодов (перекрытие брендов 82–85 %).
4. Экспериментальная оценка	точность каскада 95,5 % / 0,890 макро-F1, сквозная 93,0 %, ручной gold 86,7 %; сокращение стоимости 24× архитектурное / 580× комбинированное.
5. Демо-комплекс	<code>docker-compose</code> (Go + FastAPI + Web-UI); REST-контракт; 3 сценария запасного слоя (Sonnet 4.5 / Gemini Flash / открытая gpt-oss-120b).

Спасибо за внимание

Точность каскада **95,5 %**,
производственной цепочки **93,0 %**,
сокращение стоимости в **24×** раз.

Исходный код, ноутбук §3.3 в исполняемом
виде, демо-комплекс и текст работы — в
репозитории.

Вопросы?



github.com/Qworl/diploma

$\Leftrightarrow P_{ML}(\hat{y} | x) \geq \tau_a \wedge P_B(\hat{y} | e) \geq \theta_a$ — формализация ансамблевой оценки уверенности как конъюнкции порогов ML-классификатора (τ_a) и байесовского валидатора (θ_a).

- ▶ $\text{acc}_k^{\text{cov}} = \frac{|\hat{y}_k = y_k \wedge \hat{y}_k \neq \text{null}|}{|\hat{y}_k \neq \text{null}|}$ — точность на покрытой части (только по ячейкам, где $\hat{y} \neq \perp$);
- ▶ $\text{cov}_k = \frac{|\hat{y}_k \neq \text{null}|}{N}$ — покрытие, доля ячеек с ненулевым предсказанием;
- ▶ $\text{acc}_k^{\text{e2e}} = \text{acc}_k^{\text{cov}} \cdot \text{cov}_k$ — сквозная точность (отказ $\perp$ засчитывается как ошибка), production-realistic;
- ▶ cost_{LLM} — ожидаемая доля вызовов запасного слоя на 1000 ячеек (бюджет $\beta = 10\%$).

Связь $\text{acc}^{\text{e2e}} = \text{acc}^{\text{cov}} \cdot \text{cov}$ объясняет разрыв между только-каскадной точностью (95,5 %) и сквозной с маршрутизатором (93,0 %): часть ячеек теряется на $\text{cov} < 1$.

Гипотеза Н1: обучаемый стоимостно-чувствительный маршрутизатор превосходит статическое правило хотя бы на одном из трёх бюджетов языковой модели (25 % / 40 % / 50 %).

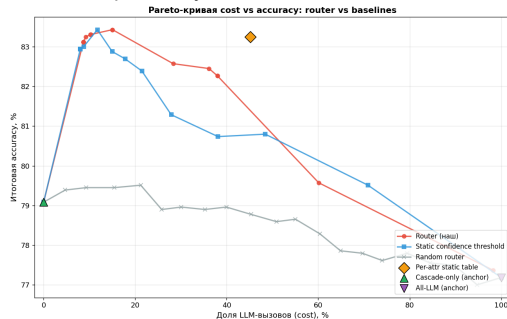
Предрегистрация: git cd9ac7a, 2026-05-13 02:11 (за 1 ч 11 мин до создания артефакта).

Поправка Бонферрони: $\alpha/3 \approx 0,0167$.

Минимально различимый эффект (MDE)
при $n = 1539$: $\approx 4,4$ пп (парный МакНемара).

Результат: $p_{\text{МакНемара}} = 0,74/0,49/1,00$ для бюджетов 25 / 40 / 50 % (все $> \alpha/3$).

Парето: обучаемый vs статический

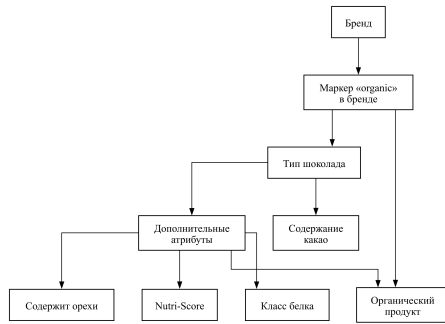


В2. Байесовский слой как валидатор плотностей

Что построено. Байесовская сеть (Hill Climb + BIC, обучена без пересечения кодов); CPT доучены на объединённой выборке (серебро + золото $\times 10$) с BDeu.

Классификатор отвергнут. Точность на сложном остатке после L2 — $\approx 52\%$, ниже majority. Переориентирован в валидатор: $P(\hat{y} | e)$ ниже 5-го перцентиля $\Rightarrow$ эскалация на L4.

Включение точечное. 1 из 20 пар: chocolate/contains_nuts — **+6,6 пп** точности при **+5,3 пп** расходов. На остальных 19 селективность сигнала $\leq$ базовой ошибки.



В3. Серебряный сигнал vs контрольная разметка

Серебряная разметка (консенсус тегов OFF + 3 языковые модели + слепая разметка Orpus) служит **обучающим** сигналом. Контрольная выборка получена по аудит-циклу.

Цикл «аудит — правки экстрактора — переобучение — измерение» воспроизведён для всех 3 категорий с приростом **+18,2, +16,3, +14,4** пп.

Калибровка уверенности — комбинация Platt + изотонической регрессии поатрибутно; ошибка ECE измеряется на отдельной выборке.

В4. Утечки и борьба с ними

Запрет утечки `categories_tags` в эмбединги.

Эмбединги Слой 2 строятся **только** из партнёрских полей: `product_name + brands + ingredients_text + quantity.categories_tags` — это **выход** обогащения, не вход; использование как признака привело бы к утечке целевой переменной.

Разбиение по непересекающимся товарным кодам (60/20/20; brand overlap train↔eval 82–85%). Бренды в обучающей, валидационной и контрольной выборках не пересекаются — гарантия обобщения на новые бренды.

Анализ по расстоянию k-NN (§3.3.7.3): точность **не падает** на удалённых от обучающей выборки товарах — монотонный рост от Q1 (82,7 %) до Q4 (91,6 %).

В5. Многоязычность

Языковой состав OFF: $\approx 40\%$ FR, 10% EN, 7% DE, остальное — прочие.

Эмбединг: `paraphrase-multilingual-mpnet-base-v2` (50+ языков, архитектура MPNet, общее семантическое пространство).

Поатрибутно-поязыковой срез ($n=3257$ ячеек, предыдущая выборка до extended regun 2026-05-27 — пересчёт перенесён в перспективу развития темы): взвешенная точность стабильна по основным языкам.

Под балансировку каскад не оптимизировался — это естественный выигрыш многоязычной базовой модели эмбедингов.

В6. Стоимость языковой модели на 1 млн SKU

Каскад делегирует языковой модели лишь 4,1 % ячеек ($n = 17\,062$) — прямая прокси к стоимости. Сокращение затрат **95,9 % (в $24\times$ раз)** относительно «прямой языковой модели на всё».

Распределение по слоям: L1 (regex высокой точности) — 21,9 %, L2 (ML) — **73,2 %**, L3 (regex низкой точности) — 0,8 %, L4 (языковая модель) — 4,1 %.

Проекция на 1 млн SKU $\times$ 20 атрибутов ($\sim 2,0 \cdot 10^7$ ячеек):

- ▶ Прямая модель — оплачены все $\sim 2,0 \cdot 10^7$ вызовов.
- ▶ Каскад — оплачены лишь $\sim 8,2 \cdot 10^5$ вызовов (4,1 %).

Топ-3 затратных атрибутов для языковой модели: pasta.grain_type (32,9 %), pasta.pasta_shape (22,8 %), cheeses.is_pdo (13,6 %).

В7. Холодный старт: электроника (§5 ноутбука)

Демонстрация переноса методики на **новую категорию без регулярных правил**.

Электроника (8 атрибутов): смартфоны — RAM, объём памяти, диагональ экрана, поддержка 5G, ...

Перенос: активны только Слой 2 (ML + эмбединги) и Слой 4 (языковая модель); Слой 1 пропускается; Слой 3 — по обучающим парам.

Результат: холодный старт работает на ограниченной обучающей выборке, что подтверждает применимость каскада за пределами продуктового домена.

В8. Где каскад слабее: precision/recall на 3 худших атрибутах

Производственная схема V6, эталон LLM-consensus (extended rerun, n=16 360).

Атрибут	Класс	n	prec	recall	F1
<i>chocolate.flavor_profile</i> — <i>micro</i> 87,5 %, <i>macro-F1</i> 0,684, <i>n</i> =1060					
	intense_bitter	397	91,1 %	98,0 %	0,944
	nutty	281	86,6 %	85,4 %	0,860
	sweet_creamy	190	80,8 %	84,2 %	0,825
	<i>salty_caramel</i> (редкий)	56	90,5 %	67,9 %	0,776
<i>pasta.is_filled</i> — <i>micro</i> 88,5 %, <i>macro-F1</i> 0,786, <i>n</i> =1225					
	false (не фаршированные)	958	87,2 %	100 %	0,931
	<i>true</i> (фаршированные)	267	100 %	47,2 %	0,641
<i>cheeses.texture</i> — <i>micro</i> 89,0 %, <i>n</i> =335					
	hard	109	95,5 %	96,3 %	0,959
	soft	97	91,3 %	97,9 %	0,945
	<i>cream</i> (редкий)	33	73,9 %	51,5 %	0,607

B9. chocolate/contains_nuts — атрибут с активным Bayes-валидатором

Единственная пара из 20, где Bayes-валидатор включён в production (см. B2).
Структура ошибок после фильтрации:

Класс	n	precision	recall	F1
false (без орехов)	749	99,1 %	93,2 %	0,961
true (с орехами)	377	87,9 %	98,4 %	0,929
Сумма	1126	micro 94,9 %		macro-F1 0,945

Почему именно Bayes-валидатор. Атрибут безопасный (аллергия — наличие орехов важно подтверждать жёстко). Распределение асимметричное: false-предсказания *критичны*, true-предсказания допускают перепроверку. Bayes-плотность ловит сомнительные false и эскалирует их в LLM — **+6,6 п. п.** категориальной точности chocolate при **+5,3 п. п.** расходов на LLM.

B10. Macro-F1 на ручной разметке: 0,843 cells-weighted

Консервативная нижняя оценка на независимом эталоне ручной разметки (Claude Opus 4, $n=688$ в-scope, $n=566$ cascade-valid).

Метрика	LLM-consensus ($n=16\,360$)	Ручной gold ($n=566$)
Cascade micro-acc	95,5 %	91,3 %
Cascade macro-F1 (cells-weighted)	0,890	0,843
Macro-precision (cells-weighted)	0,918	0,885
Macro-recall (cells-weighted)	0,881	0,843
Сквозная (производственная) acc	93,0 %	86,7 %

Разрыв 4,2 п. п. micro-acc / 0,047 F1 объясняется тремя причинами:
ручная разметка — независимый сигнал; меньший $n \rightarrow$ шире ДИ;
редкие классы под-представлены (детали в §3.3 ВКР).

B11. На что опираемся (посылки)

1. Унаследованная инфраструктура

PIM (Akeneo, Pimcore, Salsify) хранит, но не извлекает атрибуты.

2. Теоретическая основа

Многоязычные эмбединги, калиброванная классификация (Platt + isotonic), байесовские сети, prompt-инженерия (few-shot, schema-constrained).

3. Технологический стек

Python / FastAPI / Go-шлюз / React; Docker-compose; XGBoost, sentence-transformers, pgmpy; Ollama для локальной языковой модели.

4. Открытые данные

Open Food Facts (4,5 млн SKU, ~40 % FR / 10 % EN / 7 % DE) + открытые источники по электронике для холодного старта.

В12. Методы исследования

- ▶ **Системный анализ** —
декомпозиция процесса,
выявление противоречий
- ▶ **Управление требованиями** —
ФР / НФР / А, интегральный
критерий Q
- ▶ **Машинное обучение** —
эмбединги + XGBoost,
калибровка Platt + isotonic
- ▶ **Промпт-инженерия** — few-shot
со схемой ответа, селективная
эскалация
- ▶ **Предрегистрируемый
эксперимент** — МакНемара +
Бонферрони, Wilson 95 % CI
- ▶ **Итеративная разработка** —
цикл аудит → правки →
переобучение → измерение

В13. Состав требований

Функциональные (что делает)

- ▶ **F1.** Значение каждого атрибута
- ▶ **F2.** Слой-источник предсказания
- ▶ **F3.** Калиброванная уверенность $c \in [0, 1]$
- ▶ **F4.** Эскалация на запасной слой
- ▶ **F5.** Байес-валидация подозрительных решений
- ▶ **F6.** Поатрибутная настройка через конфиг

Нефункциональные (как работает)

- ▶ **N1.** Точность: $\geq 85\%$ (cascade) / $\geq 90\%$ (+LLM)
- ▶ **N2.** Стоимость: снижение $\geq 2,5\times$ vs прямой LLM
- ▶ **N3.** Производительность: ≤ 500 мс (cascade) / ≤ 5 с (LLM)
- ▶ **N4.** Многоязычность: 5 рабочих языков (fr/en/de/it/es)
- ▶ **N5.** Локальное развёртывание (open-model, без внешних API)

Архитектурные (как устроено)

A1. docker-compose; **A2.** REST-контракт между слоями; **A3.** холодный старт без правки ядра.

В14. Слой 2: XGBoost vs MLP vs LogReg vs RF

Сравнение четырёх алгоритмов на одной выборке и одинаковом эмбединге (методология §3.3.7.5).

Классификатор	Micro-acc	Macro-F1	ECE	$t_{\text{обуч, с}}$
XGBoost (производственный)	92,02 %	0,849	0,026	1062
MLP (нейронная сеть)	92,67 %	0,857	0,031	54
Логистическая регрессия	91,85 %	0,836	0,035	8
Random Forest	89,94 %	0,809	0,027	523

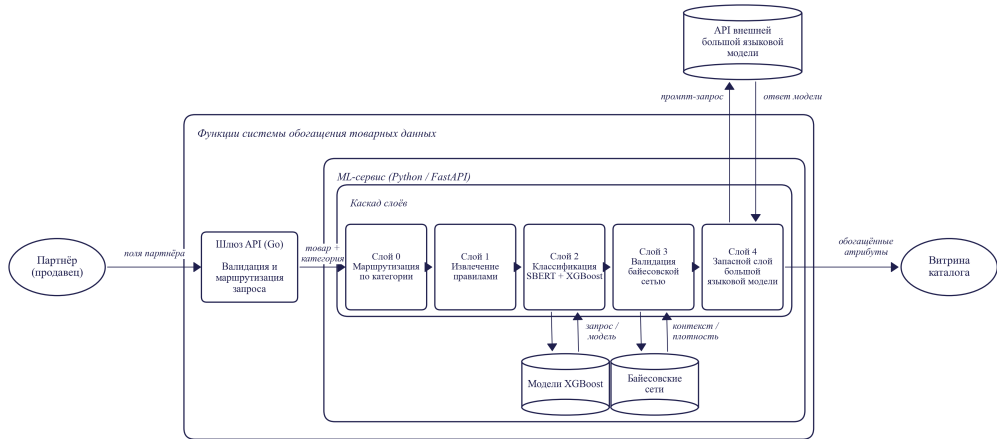
Почему XGBoost. MLP даёт чуть выше точность, но **хуже калибровку** (ECE 0,031 vs 0,026) — критично для поатрибутных порогов τ_a . RF проигрывает на 2 п. п. по точности.

Логрессия — слишком простая для разнородных категорий.

В15. Соответствие требованиям

1. **Соответствие требованиям.** Все 6 ФР, 5 НФР, 3 А реализованы; **4/4** показателя качества достигли целевых.
2. **Объём.** ~15 тыс. строк Python; 35+ модулей; 7 категориальных схем; 21 пара «категория-атрибут» (8/6/7).
3. **Демо.** `docker-compose`: React + Go + FastAPI; 5 REST-эндпоинтов; ~200 мс на запрос.
4. **Воспроизводимость.** ~70 тестов; ноутбук как исполняемый §3.3; `reproduce.sh` восстанавливает headline-цифры.

В16. Функциональная модель каскада



Каждый слой возвращает $(\hat{y}, c, \ell)$ или отказ; эскалация — только при низкой уверенности.

В17. Достоверность чисел: code-disjoint, leak-control

Достоверность

- ▶ Code-disjoint выборка: основная $n=16\,360$ + проверочная $n=566$
- ▶ **Предрегистрация** гипотезы о маршрутизаторе — `git cd9ac7a` до создания артефакта
- ▶ Wilson 95 % CI на всех headline-числах
- ▶ МакНемара + поправка Бонферрони ($\alpha/3$) для множественных сравнений
- ▶ Полное воспроизведение через `reproduce.sh`

Обоснованность. Признанные методы: sentence-transformers, XGBoost, байесовские сети, калибровка Platt + isotonic; сравнение с TXtract / MAVe / OpenTag / FrugalGPT / AutoMix.

В19. Почему один классификатор на атрибут: multitask-эксперимент

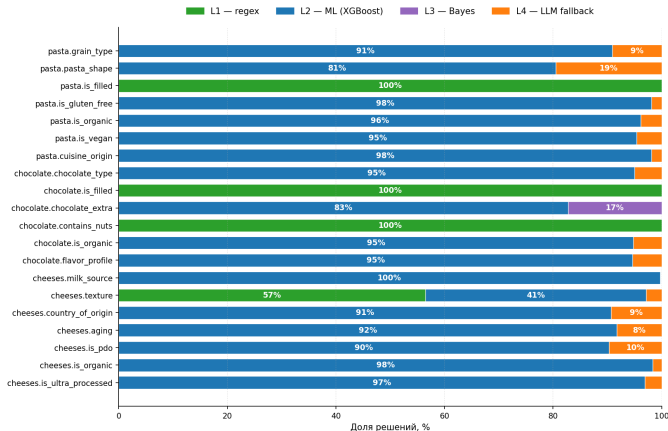
Проверено три архитектуры на 7 высоко-/средне-/слабо-коррелированных парах (3 категории):

Архитектура	Победы vs Single	Итог
MultiOutput (sklearn)	7 / 5 / 2	≈ ничья
Cartesian (декарт. пр-во)	5 / 6 / 0	—

Cartesian даёт **+0,08–0,24 macro-F1** на парах с $V \geq 0,5$ и $|A \times B| \leq 15$ (milk_source $\times$ texture: +0,235; cocoa $\times$ nuts: +0,082), но *проигрывает* на крупных пространствах (pasta_shape $\times$ is_filled, $n_{\text{class}}=22$): −0,067.

Вывод. MultiOutput технически эквивалентен независимым классификаторам (нет shared gradients). Cartesian — кандидат для пар с сильной корреляцией ($V \geq 0,5$) и узким пространством (≤ 15 классов); в производственной конфигурации оставлен per-attribute вариант как наиболее прозрачный для цикла аудит → правка →

В18. Вклад слоёв по категориям



Доля LLM по 20 атрибутам — 4,1 %; максимум на pasta.pasta_shape (19 %), pasta.grain_type (9 %). Остальное — L1 regex (22 %) + L2 ML XGBoost (73 %) + L3